

|                     |                                   |
|---------------------|-----------------------------------|
| Trạng thái          | Đã xong                           |
| Bắt đầu vào lúc     | Thứ Sáu, 23 tháng 8 2024, 3:45 PM |
| Kết thúc lúc        | Thứ Sáu, 23 tháng 8 2024, 4:17 PM |
| Thời gian thực hiện | 32 phút 17 giây                   |
| Điểm                | 7,00/7,00                         |
| Điểm                | 10,00 trên 10,00 (100%)           |



## Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement Breadth-first [search](#)

Adjacency \*BFS(int v);

where Adjacency is a structure to store list of number.

```
#include <iostream>
#include <list>
using namespace std;

class Adjacency
{
private:
    list<int> adjList;
    int size;
public:
    Adjacency() {}
    Adjacency(int V) {}
    void push(int data)
    {
        adjList.push_back(data);
        size++;
    }
    void print()
    {
        for (auto const &i : adjList)
            cout << " -> " << i;
    }
    void printArray()
    {
        for (auto const &i : adjList)
            cout << i << " ";
    }
    int getSize() { return adjList.size(); }
    int getElement(int idx)
    {
        auto it = adjList.begin();
        advance(it, idx);
        return *it;
    }
};
```

And Graph is a structure to store a graph (see in your answer box)

**For example:**

| Test                                                                                                                                                                                                                                                                                         | Result      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <pre>int V = 6; int visited = 0;  Graph g(V); Adjacency* arr = new Adjacency(V); int edge[][2] = {{0,1},{0,2},{1,3},{1,4},{2,4},{3,4},{3,5},{4,5}};  for(int i = 0; i &lt; 8; i++) {     g.addEdge(edge[i][0], edge[i][1]); }  arr = g.BFS(visited); arr-&gt;printArray(); delete arr;</pre> | 0 1 2 3 4 5 |
| <pre>int V = 6; int visited = 2;  Graph g(V); Adjacency* arr = new Adjacency(V); int edge[][2] = {{0,1},{0,2},{1,3},{1,4},{2,4},{3,4},{3,5},{4,5}};  for(int i = 0; i &lt; 8; i++) {     g.addEdge(edge[i][0], edge[i][1]); }  arr = g.BFS(visited); arr-&gt;printArray(); delete arr;</pre> | 2 0 4 1 3 5 |

Answer: (penalty regime: 0, 0, 5, 10, ... %)

Reset answer

```
1 #include <iostream>
2 #include <list>
3 #include <queue>
4 using namespace std;
5
6 class Graph {
7     int V;           // Số lượng đỉnh
8     Adjacency *adj;   // Mảng của danh sách kề
9
10 public:
11     Graph(int V) {
12         this->V = V;
13         adj = new Adjacency[V];
14     }
15
16     void addEdge(int v, int w) {
17         adj[v].push(w);
18         adj[w].push(v); // Vì đây là đồ thị vô hướng, nên cần thêm cả hai chiều
19     }
20
21     Adjacency* BFS(int startVertex) {
22         vector<bool> visited(V, false); // Đánh dấu tất cả các đỉnh chưa được thăm
23         queue<int> q;
24         Adjacency* result = new Adjacency();
25
26         visited[startVertex] = true;
27         q.push(startVertex);
28
29         while (!q.empty()) {
30             int v = q.front();
```

```

31         result->push(v); // Thêm đỉnh vào kết quả BFS
32         q.pop();
33
34         for (int i = 0; i < adj[v].getSize(); ++i) {
35             int neighbor = adj[v].getElement(i);
36             if (!visited[neighbor]) {
37                 visited[neighbor] = true;
38                 q.push(neighbor);
39             }
40         }
41     }
42
43     return result;
44 }
45 };

```

|   | Test                                                                                                                                                                                                                                                                                           | Expected    | Got         |   |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------|---|
| ✓ | <pre> int V = 6; int visited = 0;  Graph g(V); Adjacency* arr = new Adjacency(V); int edge[][2] = {{0,1},{0,2},{1,3},{1,4},{2,4},{3,4},{3,5},{4,5}};  for(int i = 0; i &lt; 8; i++) {     g.addEdge(edge[i][0], edge[i][1]); }  arr = g.BFS(visited); arr-&gt;printArray(); delete arr; </pre> | 0 1 2 3 4 5 | 0 1 2 3 4 5 | ✓ |
| ✓ | <pre> int V = 6; int visited = 2;  Graph g(V); Adjacency* arr = new Adjacency(V); int edge[][2] = {{0,1},{0,2},{1,3},{1,4},{2,4},{3,4},{3,5},{4,5}};  for(int i = 0; i &lt; 8; i++) {     g.addEdge(edge[i][0], edge[i][1]); }  arr = g.BFS(visited); arr-&gt;printArray(); delete arr; </pre> | 2 0 4 1 3 5 | 2 0 4 1 3 5 | ✓ |



|   | Test                                                                                                                                                                                                                                                                                                                         | Expected                     | Got                          |   |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------|---|
| ✓ | <pre> int V = 8, visited = 5;  Graph g(V); Adjacency *arr; int edge[][2] = {{0,1}, {0,2}, {0,3}, {0,4}, {1,2}, {2,5}, {2,6}, {4,6}, {6,7}}; for(int i = 0; i &lt; 9; i++) { \tg.addEdge(edge[i][0], edge[i][1]); }  // g.printGraph(); // cout &lt;&lt; endl; arr = g.BFS(visited); arr-&gt;printArray(); delete arr; </pre> | <pre> 5 2 0 1 6 3 4 7 </pre> | <pre> 5 2 0 1 6 3 4 7 </pre> | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



## Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Given a graph represented by an adjacency-list `edges`.

**Request:** Implement function:

```
int connectedComponents(vector<vector<int>>& edges);
```

Where `edges` is the adjacency-list representing the graph (this list has between 0 and 1000 lists). This function returns the number of connected components of the graph.

**Example:**

Given a adjacency-list: `[[1], [0, 2], [1], [4], [3], []]`

There are 3 connected components: `[0, 1, 2], [3, 4], [5]`

**Note:**

In this exercise, the libraries `iostream`, `string`, `cstring`, `climits`, `utility`, `vector`, `list`, `stack`, `queue`, `map`, `unordered_map`, `set`, `unordered_set`, `functional`, `algorithm` has been included and `namespace std` are used. You can write helper functions and classes. Importing other libraries is allowed, but not encouraged, and may result in unexpected errors.

**For example:**

| Test                                                                                                                                             | Result |
|--------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre>vector&lt;vector&lt;int&gt;&gt; graph {     {1},     {0, 2},     {1, 3},     {2},     {} }; cout &lt;&lt; connectedComponents(graph);</pre> | 2      |

**Answer:** (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
1 void DFS(int node, vector<vector<int>>& edges, vector<bool>& visited) {
2     visited[node] = true;
3
4     for (int neighbor : edges[node]) {
5         if (!visited[neighbor]) {
6             DFS(neighbor, edges, visited);
7         }
8     }
9 }
10 int connectedComponents(vector<vector<int>>& edges) {
11     int n = edges.size();
12     vector<bool> visited(n, false);
13     int count = 0;
14
15     for (int i = 0; i < n; ++i) {
16         if (!visited[i]) {
17             DFS(i, edges, visited);
18             count++;
19         }
20     }
21
22     return count;
23 }
```

|   | Test                                                                                                                                                                         | Expected | Got |   |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----|---|
| ✓ | <pre>vector&lt;vector&lt;int&gt;&gt; graph {<br/>  \t{1},<br/>  \t{0, 2},<br/>  \t{1, 3},<br/>  \t{2},<br/>  \t{}<br/>};<br/>cout &lt;&lt; connectedComponents(graph);</pre> | 2        | 2   | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



## Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Implement Depth-first [search](#)

Adjacency \*DFS(int v);

where Adjacency is a structure to store list of number.

```
#include <iostream>
#include <list>
using namespace std;

class Adjacency
{
private:
    list<int> adjList;
    int size;
public:
    Adjacency() {}
    Adjacency(int V) {}
    void push(int data)
    {
        adjList.push_back(data);
        size++;
    }
    void print()
    {
        for (auto const &i : adjList)
            cout << " -> " << i;
    }
    void printArray()
    {
        for (auto const &i : adjList)
            cout << i << " ";
    }
    int getSize() { return adjList.size(); }
    int getElement(int idx)
    {
        auto it = adjList.begin();
        advance(it, idx);
        return *it;
    }
};
```

And Graph is a structure to store a graph (see in your answer box)

**For example:**



| Test                                                                                                                                                                                                                                                                                                                           | Result          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <pre> int V = 8, visited = 0;  Graph g(V); Adjacency *arr; int edge[][2] = {{0,1}, {0,2}, {0,3}, {0,4}, {1,2}, {2,5}, {2,6}, {4,6}, {6,7}}; for(int i = 0; i &lt; 9; i++) {     g.addEdge(edge[i][0], edge[i][1]); }  // g.printGraph(); // cout &lt;&lt; endl; arr = g.DFS(visited); arr-&gt;printArray(); delete arr; </pre> | 0 1 2 5 6 4 7 3 |

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```

1 | class Graph
2 | {
3 | private:
4 |     int V;
5 |     Adjacency *adj;
6 |
7 |     void DFSUtil(int v, bool visited[], Adjacency* result) {
8 |         visited[v] = true;
9 |         result->push(v);
10 |
11 |         for (int i = 0; i < adj[v].getSize(); ++i) {
12 |             int neighbor = adj[v].getElement(i);
13 |             if (!visited[neighbor]) {
14 |                 DFSUtil(neighbor, visited, result);
15 |             }
16 |         }
17 |     }
18 |
19 | public:
20 |     Graph(int V)
21 |     {
22 |         this->V = V;
23 |         adj = new Adjacency[V];
24 |     }
25 |
26 |     void addEdge(int v, int w)
27 |     {
28 |         adj[v].push(w);
29 |         adj[w].push(v);
30 |     }
31 |
32 |     void printGraph()
33 |     {
34 |         for (int v = 0; v < V; ++v)
35 |         {
36 |             cout << "\nAdjacency list of vertex " << v << "\nhead ";
37 |             adj[v].print();
38 |         }
39 |     }
40 |
41 |     Adjacency *DFS(int v)
42 |     {
43 |         Adjacency* result = new Adjacency();
44 |         bool *visited = new bool[V];
45 |         for(int i = 0; i < V; i++)
46 |             visited[i] = false;
47 |
48 |         DFSUtil(v, visited, result);

```

```
48         DFSUtil(v, visited, result);
49
50         delete[] visited;
51         return result;
52     }
53 };
```

|   | Test                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Expected                              | Got                                   |   |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------|---|
| ✓ | <div>int V = 8, visited = 0;</div> <div>Graph g(V);</div> <div>Adjacency *arr;</div> <div>int edge[][2] = {{0,1}, {0,2}, {0,3}, {0,4}, {1,2}, {2,5}, {2,6}, {4,6}, {6,7}};</div> <div>for(int i = 0; i &lt; 9; i++)</div> <div>{</div> <div>    \tg.addEdge(edge[i][0], edge[i][1]);</div> <div>}</div> <div><br/></div> <div>// g.printGraph();</div> <div>// cout &lt;&lt; endl;</div> <div>arr = g.DFS(visited);</div> <div>arr-&gt;printArray();</div> <div>delete arr;</div> | <div>0 1 2 5 6 4 7</div> <div>3</div> | <div>0 1 2 5 6 4 7</div> <div>3</div> | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



## Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Given a graph and a source vertex in the graph, find shortest paths from source to destination vertice in the given graph using Dijkstra's algorithm.

Following libraries are included: iostream, vector, algorithm, climits, [queue](#)

For example:

| Test                                                                                                                                                                                                                                                                                                                                  | Result |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre>int n = 6; int init[6][6] = {     {0, 10, 20, 0, 0, 0},     {10, 0, 0, 50, 10, 0},     {20, 0, 0, 20, 33, 0},     {0, 50, 20, 0, 20, 2},     {0, 10, 33, 20, 0, 1},     {0, 0, 0, 2, 1, 0} };  int** graph = new int*[n]; for (int i = 0; i &lt; n; ++i) {     graph[i] = init[i]; }  cout &lt;&lt; Dijkstra(graph, 0, 1);</pre> | 10     |

Answer: (penalty regime: 0 %)

Reset answer

```
1 int Dijkstra(int** graph, int src, int dst) {
2     int n = 6; // Bạn có thể điều chỉnh kích thước n theo đồ thị của bạn
3     vector<int> dist(n, INT_MAX); // Khoảng cách từ src đến tất cả các đỉnh, khởi tạo bằng vô cùng
4     vector<bool> visited(n, false); // Mảng đánh dấu các đỉnh đã được xử lý
5     dist[src] = 0; // Khoảng cách từ src đến chính nó là 0
6
7     // Hàng đợi ưu tiên lưu cặp (khoảng cách, đỉnh)
8     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
9     pq.push({0, src}); // Đẩy đỉnh nguồn vào hàng đợi
10
11     while (!pq.empty()) {
12         int u = pq.top().second; // Lấy đỉnh có khoảng cách nhỏ nhất chưa được xử lý
13         pq.pop();
14
15         if (visited[u]) continue; // Nếu đỉnh này đã được xử lý, bỏ qua
16         visited[u] = true;
17
18         for (int v = 0; v < n; v++) {
19             if (graph[u][v] != 0 && !visited[v] && dist[u] + graph[u][v] < dist[v]) {
20                 dist[v] = dist[u] + graph[u][v]; // Cập nhật khoảng cách nếu tìm thấy đường đi ngắn hơn
21                 pq.push({dist[v], v}); // Đẩy đỉnh kề vào hàng đợi với khoảng cách mới
22             }
23         }
24     }
25
26     return dist[dst]; // Trả về khoảng cách ngắn nhất từ src đến dst
27 }
```

|   | Test                                                                                                                                                                                                                                                                                                                    | Expected | Got |   |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----|---|
| ✓ | <pre>int n = 6; int init[6][6] = { \t{0, 10, 20, 0, 0, 0}, \t{10, 0, 0, 50, 10, 0}, \t{20, 0, 0, 20, 33, 0}, \t{0, 50, 20, 0, 20, 2}, \t{0, 10, 33, 20, 0, 1}, \t{0, 0, 0, 2, 1, 0} };  int** graph = new int*[n]; for (int i = 0; i &lt; n; ++i) { \tgraph[i] = init[i]; }  cout &lt;&lt; Dijkstra(graph, 0, 1);</pre> | 10       | 10  | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



## Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

The relationship between a group of people is represented by an adjacency-list `friends`. If `friends[u]` contains `v`, `u` and `v` are friends. Friendship is a two-way relationship. Two people are in a friend group as long as there is some path of mutual friends connecting them.

**Request:** Implement function:

```
int numberOfFriendGroups(vector<vector<int>>& friends);
```

Where `friends` is the adjacency-list representing the friendship (this list has between 0 and 1000 lists). This function returns the number of friend groups.

**Example:**

Given a adjacency-list: `[[1], [0, 2], [1], [4], [3], []]`

There are 3 friend groups: `[0, 1, 2], [3, 4], [5]`

**Note:**

In this exercise, the libraries `iostream`, `string`, `cstring`, `climits`, `utility`, `vector`, `list`, `stack`, `queue`, `map`, `unordered_map`, `set`, `unordered_set`, `functional`, `algorithm` have been included and `namespace std` is used. You can write helper functions and class. Importing other libraries is allowed, but not encouraged.

**For example:**

| Test                                                                                                                                                    | Result |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <pre>vector&lt;vector&lt;int&gt;&gt; graph {     {1},     {0, 2},     {1},     {4},     {3},     {} }; cout &lt;&lt; numberOfFriendGroups(graph);</pre> | 3      |

**Answer:** (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
1 #include <vector>
2 using namespace std;
3
4 void DFS(int node, vector<vector<int>>& friends, vector<bool>& visited) {
5     visited[node] = true;
6     for (int neighbor : friends[node]) {
7         if (!visited[neighbor]) {
8             DFS(neighbor, friends, visited);
9         }
10    }
11 }
12
13 int numberOfFriendGroups(vector<vector<int>>& friends) {
14     int n = friends.size();
15     vector<bool> visited(n, false);
16     int friendGroups = 0;
17
18     for (int i = 0; i < n; ++i) {
19         if (!visited[i]) {
20             DFS(i, friends, visited);
21             friendGroups++;
22         }
23     }
24 }
```

```
25     return friendGroups;
26 }
```

|   | Test                                                                                                                                                                | Expected | Got |   |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----|---|
| ✓ | <pre>vector&lt;vector&lt;int&gt;&gt; graph {     \t{1},     \t{0, 2},     \t{1},     \t{4},     \t{3},     \t{} }; cout &lt;&lt; numberOfFriendGroups(graph);</pre> | 3        | 3   | ✓ |
| ✓ | <pre>vector&lt;vector&lt;int&gt;&gt; graph {  }; cout &lt;&lt; numberOfFriendGroups(graph);</pre>                                                                   | 0        | 0   | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

Implement function to detect a cyclic in Graph

```
bool isCyclic();
```

Graph structure is defined in the initial code.

For example:

| Test                                                                                                                                                                                                                                                                                                    | Result                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <pre>DirectedGraph g(8); int edge[][2] = {{0,6}, {1,2}, {1,4}, {1,6}, {3,0}, {3,4}, {5,1}, {7,0}, {7,1}};  for(int i = 0; i &lt; 9; i++)     g.addEdge(edge[i][0], edge[i][1]);  if(g.isCyclic())     cout &lt;&lt; "Graph contains cycle"; else     cout &lt;&lt; "Graph doesn't contain cycle";</pre> | Graph doesn't contain cycle |

Answer: (penalty regime: 0, 0, 5, ... %)

Reset answer

```
1 | #include <iostream>
2 | #include <vector>
3 | #include <list>
4 | using namespace std;
5 |
6 | class DirectedGraph {
7 |     int V;
8 |     vector<list<int>> adj;
9 |
10 |     bool isCyclicUtil(int v, vector<bool>& visited, vector<bool>& recStack) {
11 |         if (!visited[v]) {
12 |             // Mark the current node as visited and part of the recursion stack
13 |             visited[v] = true;
14 |             recStack[v] = true;
15 |
16 |             // Recur for all the vertices adjacent to this vertex
17 |             for (auto neighbor : adj[v]) {
18 |                 if (!visited[neighbor] && isCyclicUtil(neighbor, visited, recStack))
19 |                     return true;
20 |                 else if (recStack[neighbor])
21 |                     return true;
22 |             }
23 |         }
24 |         recStack[v] = false; // Remove the vertex from recursion stack
25 |         return false;
26 |     }
27 |
28 | public:
29 |     DirectedGraph(int V) {
30 |         this->V = V;
31 |         adj = vector<list<int>>(V, list<int>());
32 |     }
33 |
34 |     void addEdge(int v, int w) {
35 |         adj[v].push_back(w);
36 |     }
37 | }
```

```
38 bool isCyclic() {
39     vector<bool> visited(V, false);
40     vector<bool> recStack(V, false);
41
42     for (int i = 0; i < V; i++) {
43         if (isCyclicUtil(i, visited, recStack))
44             return true;
45     }
46
47     return false;
48 }
49 };
```

|   | Test                                                                                                                                                                                                                                                                                                                                                  | Expected                    | Got                         |   |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|-----------------------------|---|
| ✓ | <div>DirectedGraph g(8);<br/>int edge[][2] = {{0,6}, {1,2}, {1,4}, {1,6}, {3,0}, {3,4},<br/>{5,1}, {7,0}, {7,1}};<br/><br/>for(int i = 0; i &lt; 9; i++)<br/>    \tg.addEdge(edge[i][0], edge[i][1]);<br/><br/>if(g.isCyclic())<br/>    \tcout &lt;&lt; "Graph contains cycle";<br/>else<br/>    \tcout &lt;&lt; "Graph doesn't contain cycle";</div> | Graph doesn't contain cycle | Graph doesn't contain cycle | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.





Câu hỏi 7

Đúng

Đạt điểm 1,00 trên 1,00

Implement **topologicalSort** function on a graph. (Ref [here](#))

```
void topologicalSort();
```

where Adjacency is a structure to store list of number. Note that, the vertex index starts from 0. **To match the given answer, please always traverse from 0 when performing the [sorting](#).**

```
#include <iostream>
#include <list>
using namespace std;

class Adjacency
{
private:
    list<int> adjList;
    int size;
public:
    Adjacency() {}
    Adjacency(int V) {}
    void push(int data)
    {
        adjList.push_back(data);
        size++;
    }
    void print()
    {
        for (auto const &i : adjList)
            cout << " -> " << i;
    }
    void printArray()
    {
        for (auto const &i : adjList)
            cout << i << " ";
    }
    int getSize() { return adjList.size(); }
    int getElement(int idx)
    {
        auto it = adjList.begin();
        advance(it, idx);
        return *it;
    }
};
```

And Graph is a structure to store a graph (see in your answer box). You could write one or more helping functions.

For example:

| Test                                                                                                                                                            | Result      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Graph g(6);<br>g.addEdge(5, 2);<br>g.addEdge(5, 0);<br>g.addEdge(4, 0);<br>g.addEdge(4, 1);<br>g.addEdge(2, 3);<br>g.addEdge(3, 1);<br><br>g.topologicalSort(); | 5 4 2 3 1 0 |

Answer: (penalty regime: 0, 0, 5, 10, ... %)

Reset answer

```
1 #include <iostream>
2 #include <vector>
3 #include <stack>
4
5 using namespace std;
6
7 class Graph {
8     int V;
9     Adjacency* adj;
10
11     void topologicalSortUtil(int v, vector<bool>& visited, stack<int>& Stack) {
12         // Đánh dấu đỉnh hiện tại là đã thăm
13         visited[v] = true;
14
15         // Lặp qua tất cả các đỉnh kề của đỉnh hiện tại
16         for (int i = 0; i < adj[v].getSize(); ++i) {
17             int neighbor = adj[v].getElement(i);
18             if (!visited[neighbor]) {
19                 topologicalSortUtil(neighbor, visited, Stack);
20             }
21         }
22
23         // Đẩy đỉnh hiện tại vào ngăn xếp
24         Stack.push(v);
25     }
26
27 public:
28     Graph(int V) {
29         this->V = V;
30         adj = new Adjacency[V];
31     }
32
33     void addEdge(int v, int w) {
34         adj[v].push(w);
35     }
36
37     void topologicalSort() {
38         stack<int> Stack;
39         vector<bool> visited(V, false);
40
41         // Gọi hàm topologicalSortUtil cho tất cả các đỉnh chưa được thăm
42         for (int i = 0; i < V; i++) {
43             if (!visited[i]) {
44                 topologicalSortUtil(i, visited, Stack);
45             }
46         }
47
48         // In ra nội dung ngăn xếp (thứ tự sắp xếp topo)
49         while (!Stack.empty()) {
50             cout << Stack.top() << " ";
51             Stack.pop();
52         }
53     }
54 };
```

|   | Test                                                                                                                                               | Expected    | Got         |   |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------|---|
| ✓ | <pre>Graph g(6); g.addEdge(5, 2); g.addEdge(5, 0); g.addEdge(4, 0); g.addEdge(4, 1); g.addEdge(2, 3); g.addEdge(3, 1);  g.topologicalSort();</pre> | 5 4 2 3 1 0 | 5 4 2 3 1 0 | ✓ |

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

